Mathematics Internal Assessment

Title: Evaluation of the gradient descent algorithm with the example of linear regression
Research Question: How can the gradient descent algorithm be used for finding the optimal slope and intercept for a linear regression line?

Page Count: 19

1 Introduction

The topic of my exploration is the application of the gradient descent algorithm in linear regression and the algorithm's limitations. The concept of gradient descent originates from machine learning, a subfield of computer science that aims to use data and algorithms to replicate human learning [1]. Gradient descent is an iterative optimisation algorithm for finding a local minimum of a differentiable function [2]. In machine learning cases, gradient descent is used to minimize a so-called loss function, which is a method of measuring how well a particular model (such as linear regression) predicts the data.

I aim to explore and assess the working principles of the gradient descent algorithm with the help of a linear regression problem. My exploration 'mainly focuses on two topics in mathematics that are calculus and statistics. I use partial derivatives to find the slope of a two-variable function and I use linear regression as a convenient practical example for explaining gradient descent and showing its real-world application.

The choice of this topic came from my curiosity about machine learning and its branch known as artificial intelligence. I think that these fields seem extraordinarily difficult for the majority of people, so I was excited to explore how such algorithms work in detail. Also, I was interested in how my calculator finds the local minima and maxima of functions numerically. After some research, I learned about numerical analysis which gradient descent is very closely related to.

Firstly, I am going to set a basic linear regression problem of predicting a certain value of the dependent variable with a regression line. Then, I will derive the loss function that will allow me to quantify the error for certain parameters for the regression line. Then, I will derive and explain in detail the gradient descent update rule using partial derivatives and the loss function. Using the update rule and some programming, I will find approximate the optimal parameters for my regression line. Then, I will finish with the limitations of the gradient descent algorithm. For plotting functions I will use Python programming language [3] and its extension Scikit-learn [4].

2 Exploration

2.1 Linear Regression

Suppose I studied 10 hours per week for my mathematics exam, and now I want to predict my grade. I collect data from my peers and past exams which is presented below. Note that this data is fully made up by me and is used only for context and ease of understanding.

Hours studied, x	1	3	4	8	14	9	12
Exam grade, y	1	2	3	4	5	6	7

Table 1: Hours studied per week and the grade that a class of students received for a mathematics test (made up by me)

To estimate my exam grade I can model the relationship between the independent variable (hours studied) and the dependent variable (exam grade) with linear regression

[5]. This model is represented by a sloped straight regression line $y = mx + c$ with slope m and intercept c. The slope and intercept which are the parameters of the model are selected so that the line provides the best accurate estimate of the data points. I will use the optimal regression line to estimate the exam grade based on the hours studied for the exam by substituting 10 for x which will yield the predicted exam grade. Hence, my goal is to find the optimal parameters m and c for this straight line y.

The best fit by eye is one of the easiest methods for fitting a line to a dataset. It is done by plotting the mean point $(\overline{x}, \overline{y})$ and fitting a line through it so that approximately the same number of data points is above and below it. Although this method is effective, it is not precise. Furthermore, to showcase the difference between an optimal and a suboptimal combination of m and c at the end of my exploration, I purposefully decided to choose $m = 1$ and $c = 3$ which do not fit the data points well. Also, integer parameters will simplify the calculations making it easier to follow my exploration. Thus, my initial regression line is $\hat{y} = x - 3$ plotted below. Note that $\hat{y}$ (y hat) means the prediction of the actual grade y.

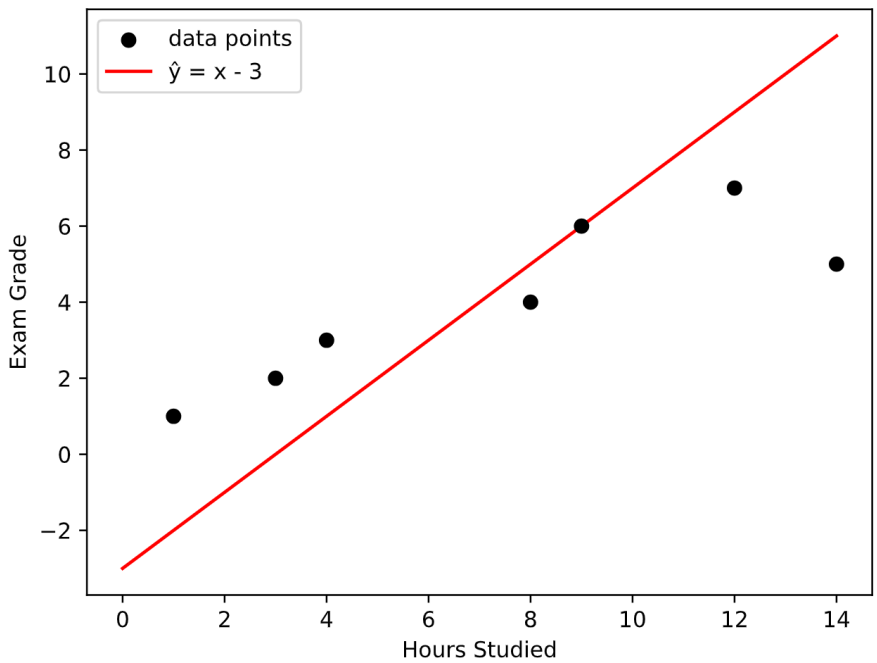

Figure 1: The plot of my regression line with slope 1 and intercept -3 and the data points

Reflection 1

As previously stated, it is clear that the line does not follow the data points accurately. Moreover, it predicts a negative grade for less than 3 hours studied which is of course not possible to get on a real exam. However, this shows that I can definitely improve the parameters. For now, to measure the "correctness" of this and any other set of parameters m and c I will find the residuals, or the errors between observed data points (black) and predicted data points (red line):

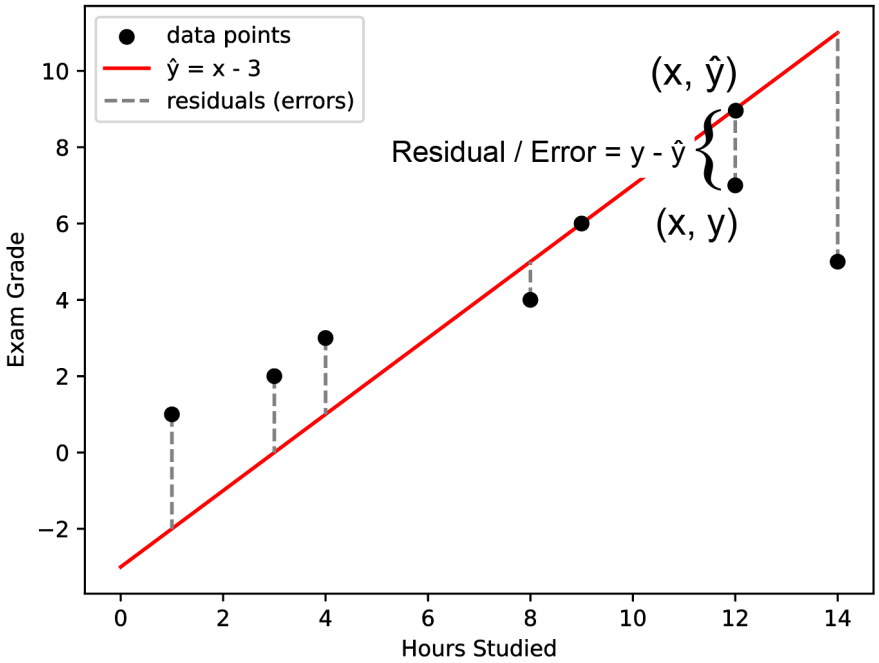

Figure 2: The plot of my regression line with residuals

The residuals can be represented as the distance between y and $\hat{y}$, or $y_i - \hat{y}_i$ where y_i is ith actual observation and $\hat{y}_i$ is ith prediction of the grade for specific number of hours studied. In the ideal case where the predicted grade equals the observed grade, or when $\hat{y}_i = y_i$, all our data points lie on the regression line and the sum of the residuals is zero. So, we need to somehow minimize these residuals by modifying the slope m and intercept c. Thus we will get a variety of different line fits with various errors. However, to find the case where these total residuals are the smallest we need to define a loss function or a function that measures the error between a model's prediction and the actual target output [6].

I noticed that some residuals like the one highlighted in Figure 2 are negative because $y < \hat{y}$. This is problematic as when I will sum up the residuals, some of them will cancel out. To solve this, I can square each residual to get rid of the sign. Then I will sum up these squared residuals and find the mean by dividing the sum by the number of data points. I need the mean to find the average error for each data point which will allow me to minimize the error for all data points. Thus I get the mean squared error (MSE) which is a common loss function that is usually used in linear regression problems [7]:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{1}$$

where n is the number of data points, y_i is the ith observation and $\hat{y}_i$ is the ith prediction. In the case of my regression problem the number of data points is seven, so $n = 7$. Since $\hat{y}_i$ is a linear regression line, it is $\hat{y}_i = mx_i + c$. So, I will rewrite MSE as a two-variable function of variables m and c denoted with J:

$$J(m, c) = \frac{1}{7}\sum_{i=1}^{7}(y_i - (mx_i + c))^2 \tag{2}$$

where x_i and y_i are just sets of values from Table 1. Now I can calculate the MSE for my regression line, $\hat{y} = x - 3$ to measure how well it estimates the exam grade. As I said before, in the ideal case when $y_i = \hat{y}_i$ the errors $y_i - \hat{y}_i = 0$. So, since my model is deliberately designed to predict the exam grade poorly, the MSE should not be close to zero, and I expect it to be in the range of 5 to 15. Now I will find the errors and errors squared to calculate the MSE:

Observation (grade), y	1	2	3	4	5	6	7
Prediction, $\hat{y} = x_i - 3$	-2	0	1	5	11	6	9
Error, $y - \hat{y} = y - (x_i - 3)$	3	2	2	-1	-6	0	-2
Error squared, $(y - \hat{y})^2$	9	4	4	1	36	0	4

Table 2: Error calculations for my model $\hat{y}_i = x_i - 3$ with observations from Table 1 and predictions from Figure 1

Now I calculate the MSE by substituting errors squared in equation 2:

$$J(1,-3) = \frac{1}{7}\Big(9+4+4+1+36+0+4\Big) = \frac{58}{7} \approx 8.2857$$

The MSE is 8.2857 to 5 significant digits. As I expected, MSE is not close to zero and is in fact between 5 and 15 as I expected. Now I will try to find such m and c so that MSE is minimized. Since J is a two-variable function and its z-axis represents the MSE, there should be a minimum point which represents the minimum MSE. Let's expand the sum in equation (2) and substitute actual values for x_i and y_i from Table 1 to plot it:

$$\begin{aligned} J(m,c) = \frac{1}{7}\Big[&(1-(m\cdot 1+c))^2 + (2-(m\cdot 3+c))^2 + \\ &+ (3-(m\cdot 4+c))^2 + (4-(m\cdot 8+c))^2 + (5-(m\cdot 14+c))^2 + \\ &+ (6-(m\cdot 9+c))^2 + (7-(m\cdot 12+c))^2\Big] \end{aligned} \quad (3)$$

Now I can graph this function using software or Python programming language [3] in my case:

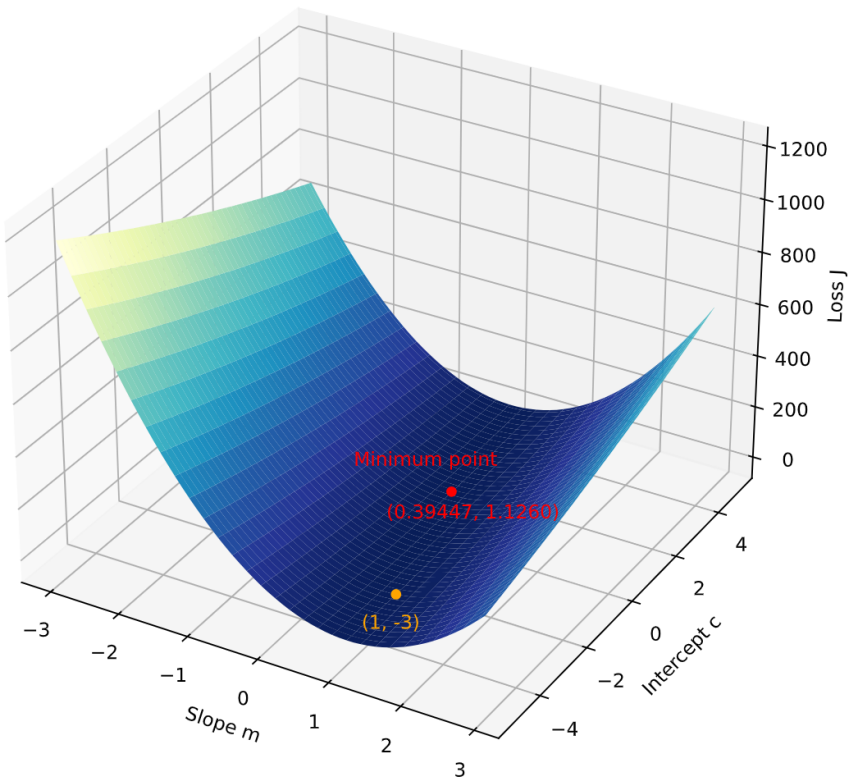

Figure 3: 3D plot of my loss function $J(m, c)$

Every point on this graph is a combination of slope and intercept that my model $\hat{y} = mx + c$ can have, and the error that this combination results in. The red dot is the minimum point that I found using an extension of Python programming language called Scikit-learn [4] which is a free software machine learning library. So, using Scikit-learn I got the optimal parameters $m = 0.39447$ and $c = 1.1260$ indicated as the red dot. With these parameters the error is minimal, so the regression line predicts the exam grade the best. The orange dot is my original parameters $m = 1$ and $c = -3$ far from the minimum point.

Reflection 2

Since a linear regression problem is relatively simple and the optimal m and c can be found analytically using the method of least squares, my aim will not be to find this point but to show that gradient descent can approximate it numerically. More complex machine learning problems typically feature loss functions with thousands and millions of parameters. 1do not have analytical solutions expressed in formulas like least squares [8],

so we need to use iterative numerical methods that can only approximate the solutions. An example of a numerical method is Euler's method of approximating solutions to first-order differential equations and gradient descent is very similar to it. For this reason, in the next section I will use gradient descent to approximate optimal parameters m and c.

2.2 Gradient Descent Algorithm

Gradient descent can be understood quite intuitively with a real-world example. Suppose you are on a hill and you are trying to get down from it. The steps of your "hill descent" algorithm would be something similar to this:

1. First find the direction that leads downhill.
2. Make a step in this direction.
3. Repeat the process until all other possible directions lead uphill.

The gradient descent algorithm follows essentially the same steps. We take an arbitrary initial position $(m, c, J(m, c))$ on our loss function and we calculate the slope at that point. The slope gives the direction we need to follow to get to the local minimum point. Then, we take a step towards this direction. So, the algorithm is the following:

1. Initialize arbitrary values for m and c.
2. Calculate the slope at point $(m, c, J(m, c))$.
3. Update the position by making a step towards minimum.
4. Repeat steps 2-3 until convergence to the minimum meaning that the slope is approximately zero.

The gradient descent algorithm is visualized below.

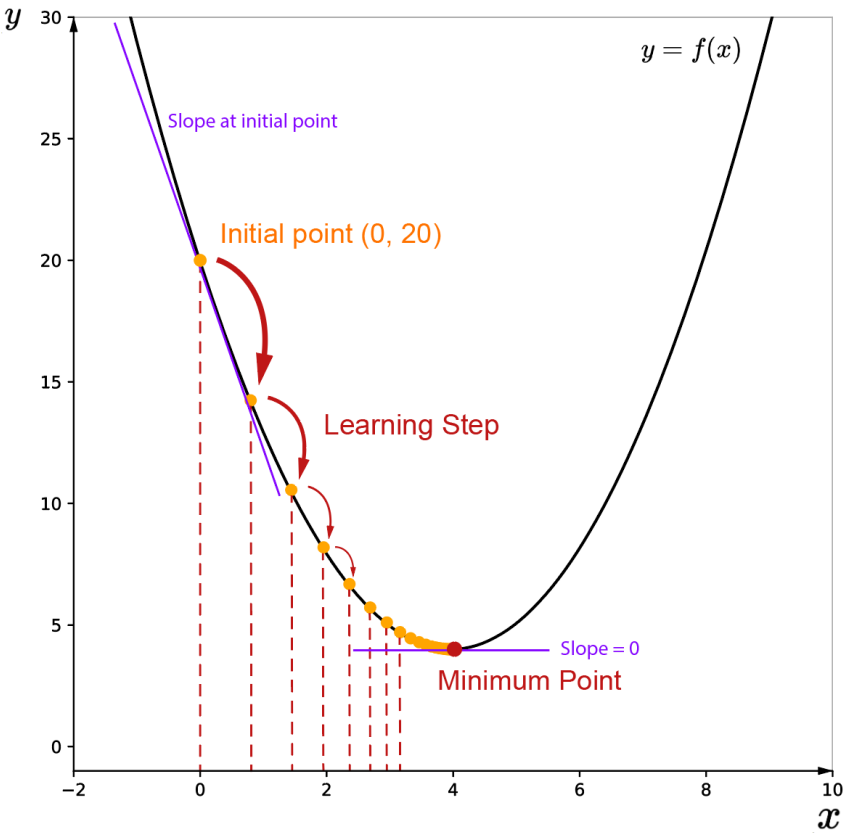

Figure 4: Visualization of gradient descent algorithm with an arbitrary function

Figure 4 illustrates how gradient descent works in practice on a continuous differentiable function $f(x)$. The initial arbitrary position is set to $(0, 20)$. Gradient descent first computes the slope at this point indicated in purple. The slope allows to find the direction that leads to the minimum and after n learning steps, or iterations, the algorithm will inevitably asymptotically converge to a point where the slope is zero. Note that the size of the learning step is proportional to the slope of the function at that point. So, the algorithm makes bigger steps when the slope of the tangent line to the function is high and makes smaller steps when the slope is closer to zero. This also suggests one of the limitations of gradient descent that I will discuss later in my exploration. Gradient descent updates the coordinates of the initial point using a special update rule that I will derive now. Using this rule we can optimize the parameters m and c in a certain number of iterations. For the update rule, we need to calculate the slope of J. The derivative of my loss function can be represented by a vector consisting of partial derivatives of the

loss function with respect to its parameters m and c:

$$\nabla J(m,c) = \begin{bmatrix} \frac{\partial J}{\partial m} \\ \frac{\partial J}{\partial c} \end{bmatrix} \tag{4}$$

A partial derivative is a derivative of a multivariable function where the function is differentiated with respect to one variable while keeping other variables constant. This gradient vector points in the direction of the fastest increase of a function [9]. This means that if we reverse its direction by putting a minus sign in front of it, it will lead to the direction of fastest decrease. So, we need to differentiate J with respect to m and c separately. Recall that J is $J(m,c) = \frac{1}{7}\sum_{i=1}^{7}(y_i - (mx_i + c))^2$. For the sake of simplicity, I will replace $y_i - (mx_i + c)$ with $Error_i$:

$$J(m,c) = \frac{1}{7}\sum_{i=1}^{7}(Error_i)^2$$

Since the sum is finite and it converges, its derivative is simply the sum of the derivatives of $Error_i$. So, first I apply chain rule $\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$ and differentiate with respect to m:

$$\frac{\partial J}{\partial m} = \frac{1}{7}\sum_{i=1}^{7}\left(2 \cdot Error_i \cdot \frac{\partial Error_i}{\partial m}\right) \tag{5}$$

Differentiating with respect to c gives:

$$\frac{\partial J}{\partial c} = \frac{1}{7}\sum_{i=1}^{7}\left(2 \cdot Error_i \cdot \frac{\partial Error_i}{\partial c}\right) \tag{6}$$

Now I need to find $\frac{\partial}{\partial m}Error_i$ and $\frac{\partial}{\partial c}Error_i$. $Error$ is just a linear function, so differentiation is quite simple. First, I differentiate $Error_i$ with respect to m keeping c constant:

$$\frac{\partial}{\partial m}Error_i = \frac{\partial}{\partial m}\big(y_i - (mx_i + c)\big) = \frac{\partial}{\partial m}\big(y_i - mx_i - c\big) = -x_i$$

Differentiating with respect to c while keeping m constant:

$$\frac{\partial}{\partial c}Error_i = \frac{\partial}{\partial c}\big(y_i - (mx_i + c)\big) = \frac{\partial}{\partial c}\big(y_i - mx_i - c\big) = -1$$

I substitute these two derivatives into equations (5) and (6):

$$\frac{\partial J}{\partial m} = \frac{1}{7}\sum_{i=1}^{7}\big(2 \cdot Error_i \cdot (-x_i)\big) = \frac{2}{7}\sum_{i=1}^{7}\Big(-x\big(y_i - (mx_i + c)\big)\Big)$$

$$\frac{\partial J}{\partial c} = \frac{1}{7}\sum_{i=1}^{7}\big(2 \cdot Error_i \cdot (-1)\big) = \frac{2}{7}\sum_{i=1}^{7}\Big(-\big(y_i - (mx_i + c)\big)\Big)$$

Now I can rewrite the gradient vector equation (4) using the derivatives I just found :

$$\nabla J(m, c) = \begin{bmatrix} \frac{2}{7}\sum_{i=1}^{7}(y_i - (mx_i + c))(-x_i) \\ \frac{2}{7}\sum_{i=1}^{7}(y_i - (mx_i + c))(-1) \end{bmatrix} \tag{7}$$

This allows me to calculate the slope of my loss function at any given point. As mentioned above, this derivative is a vector that gives the direction of the steepest ascent [9]. Since the goal of gradient descent is to minimize function J, I can multiply the vector by a negative scalar -1 to reverse its direction. So now the direction that gradient descent needs to follow is $-\nabla J(m, c)$. My first idea was to subtract this vector from the current position:

$$\begin{pmatrix} m_{new} \\ c_{new} \end{pmatrix} = \begin{pmatrix} m_{current} \\ c_{current} \end{pmatrix} - \nabla J(m_{current}, c_{current})$$

However, I noticed that if the slope is too large, the algorithm will diverge. To control the step size I can add a learning rate coefficient denoted with α. The learning rate is usually in the range from 0 to 1 and it controls the step size and the convergence rate of the algorithm [10]. Thus the new update rule looks like this:

$$\begin{pmatrix} m_{new} \\ c_{new} \end{pmatrix} = \begin{pmatrix} m_{current} \\ c_{current} \end{pmatrix} - \alpha\nabla J(m_{current}, c_{current})$$

Now when the slope of the loss function is steep the algorithm takes reasonably big steps and when the slope approaches zero, the step size becomes smaller and smaller.

Thus the final gradient descent update rule is:

$$\begin{pmatrix} m_{i+1} \\ c_{i+1} \end{pmatrix} = \begin{pmatrix} m_i \\ c_i \end{pmatrix} - \alpha \nabla J(m_i, c_i)$$

where $\begin{pmatrix} m_{i+1} \\ c_{i+1} \end{pmatrix}$ is the new position, $\begin{pmatrix} m_i \\ c_i \end{pmatrix}$ is the previous position, α is the learning rate and $\nabla J(m_i, c_i)$ is the gradient vector of J for the previous position. Now, I will use this rule to find the optimal parameters m and c for my regression line $\hat{y}_i = x_i - 3$.

First let's calculate the position after the first step. I will take the learning rate to be $\alpha = 0.01$. Using equation (7), the gradient vector of my starting position $(1, -3)$ is:

$$\nabla J(m, c) = \begin{bmatrix} \frac{\partial J}{\partial m} \\ \frac{\partial J}{\partial c} \end{bmatrix} = \begin{bmatrix} \frac{2}{7} \sum_{i=1}^{7} (y_i - (x_i - 3))(-x_i) \\ \frac{2}{7} \sum_{i=1}^{7} (y_i - (x_i - 3))(-1) \end{bmatrix}$$

By substituting values from Table 1 for x_i and y_i I get the slope for m:

$$\begin{aligned} \frac{\partial J}{\partial m} = \frac{2}{7}\big((1 - (1-3))(-1) + (2 - (3-3))(-3) + (3 - (4-3))(-4) \\ + (4 - (8-3))(-8) + (5 - (14-3))(-14) + (6 - (9-3))(-9) + \\ (7 - (12-3))(-12)\big) = 28.286 \end{aligned}$$

and the slope for c:

$$\begin{aligned} \frac{\partial J}{\partial c} = \frac{2}{7}\big((1 - (1-3))(-1) + (2 - (3-3))(-1) + (3 - (4-3))(-1) + \\ + (4 - (8-3))(-1) + (5 - (14-3))(-1) + (6 - (9-3))(-1) + \\ + (7 - (12-3))(-1)\big) = 0.57143 \end{aligned}$$

Thus the coordinates after the first iteration to 5 significant digits are:

$$\begin{pmatrix} m_{i+1} \\ c_{i+1} \end{pmatrix} = \begin{pmatrix} 1 \\ -3 \end{pmatrix} - 0.01 \begin{pmatrix} 28.286 \\ 0.57143 \end{pmatrix} = \begin{pmatrix} 1 \\ -3 \end{pmatrix} - \begin{pmatrix} 0.28286 \\ 0.0057143 \end{pmatrix} = \begin{pmatrix} 0.71714 \\ -3.0057 \end{pmatrix}$$

Then the algorithm continues in the same manner but now $m_i = 0.71714$ and $c_i = -3.0057$. Since updating positions hundreds of times is very repetitive, I implemented gradient descent in Python programming language [3] which automated the calculations. Having run the gradient descent algorithm for a 1000 iterations [Appendix A], I selected the positions only for certain iterations:

Iteration	m	c	J (MSE loss)
0	1	-3	8.2857
1	0.71714	-3.0057	6.1458
2	0.84809	-2.9701	5.6252
⋮	⋮	⋮	⋮
100	0.63601	-1.2852	2.4871
⋮	⋮	⋮	⋮
500	0.42213	0.84991	0.92142
⋮	⋮	⋮	⋮
1000	0.39631	1.1076	0.90071

Table 3: m and c values for different iterations (to 5 s.d.)

It can be seen that the MSE decreases with iterations which is the whole point of gradient descent. I also noticed that in one iteration the progress done by gradient descent is minimal which suggests that the algorithm is not efficient. It also seemed quite strange that on the second iteration the value of m increased from 0.71714 to 0.84809 but at the end decreased to 0.39631. To understand this, I decided to look at the contour plot of the loss function. A contour plot is a graphical way to represent a three-dimensional surface on a two-dimensional plane [11] and it can be imagined as looking at my loss function in Figure 3 from above:

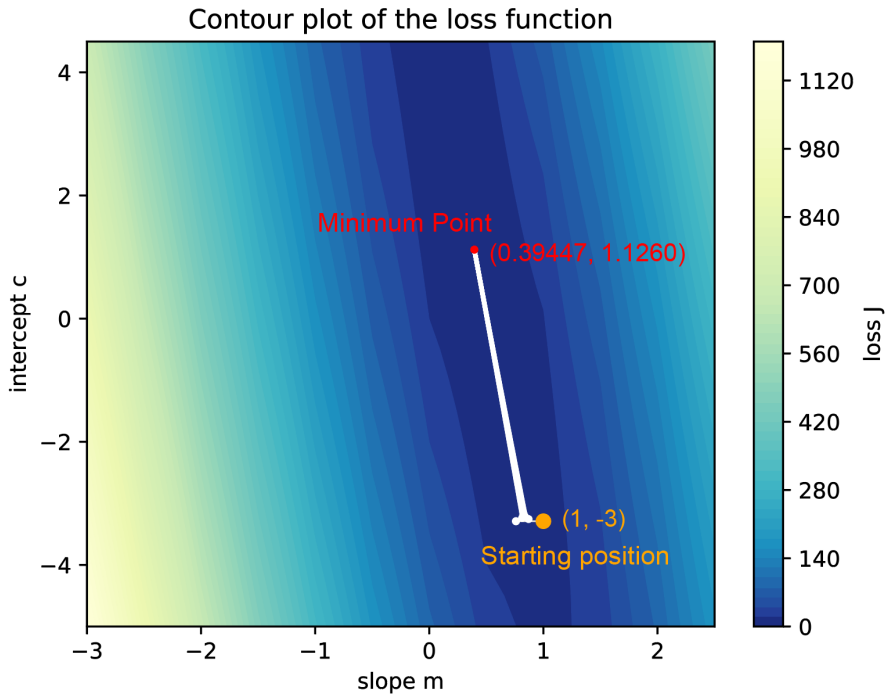

Figure 5: Path of gradient descent with a learning rate of 0.01 for my loss function J from the starting position $(1, -3)$ after 1000 iterations

It is hard to see it in the plot but the path follows a zig-zag pattern slowly converging to the minimum point. In this plot, the loss is represented with colour as seen in the colour bar on the right. Brighter colours represent higher error, and darker colours represent lower error. After around 1000 iterations gradient descent converged to a minimum at $(0.39631, 1.1076)$ correct to five significant digits. The point it converged to is indistinguishable from the red minimum point, so I did not plot it. Overall, the gradient descent algorithm has optimized m and c parameters for my regression line $\hat{y} = mx + c$ and now the optimized regression line is:

$$\hat{y}_i^* = 0.39631x + 1.1076$$

Reflection 3

Surely, these parameters do not equal the ideal ones but the percentage error between them is less than 2%:

$$\frac{1.1260 - 1.1076}{1.1260} \cdot 100 = 1.63\%$$

$$\frac{|0.39447 - 0.39631|}{0.39447} \cdot 100 = 0.46\%$$

This means that the optimized parameters are quite accurate but with more iterations the percentage error can be decreased even further. I am going to plot this line as in Figure 1 to see how well it fits the data.

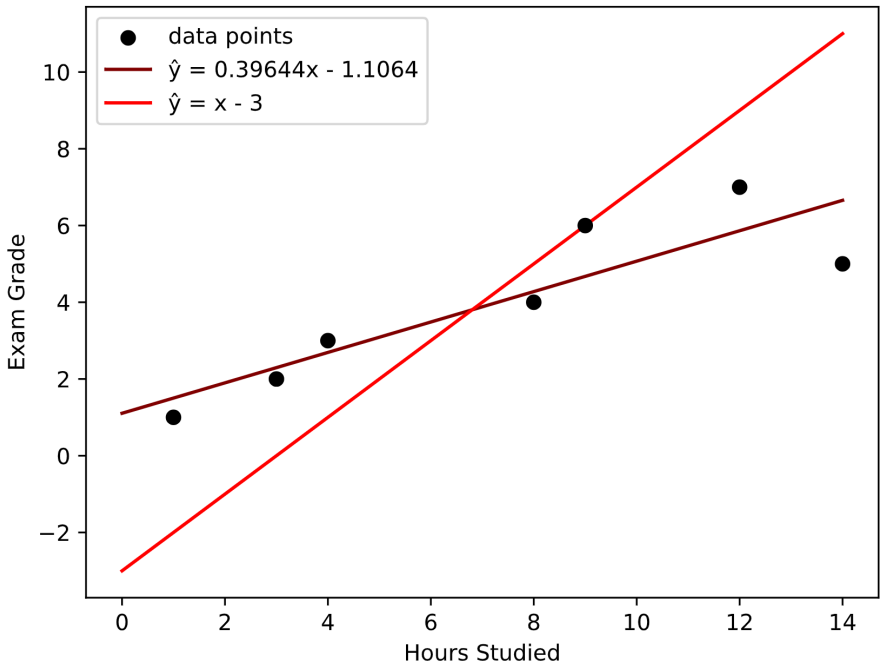

Figure 6: Regression line for my data from Table 1 with my initial fit and the fit optimized with gradient descent

The bright red line is my original fit $\hat{y}_i = x - 3$ which was deliberately made to predict the data poorly. The optimized fit is shown in dark red and it is evident that it follows the data a lot better. The MSE for my initial fit was 8.2857 and gradient descent reduced it to only 0.90071. Now I can finally estimate my exam grade considering I studied 10 hours for the exam. Thus, the grade predicted by the model with parameters optimised by my gradient descent algorithm is:

$$\hat{y}^* = 0.39631 \cdot 10 + 1.1076 = 5.0707$$

And the grade predicted by the model with optimal parameters is:

$$\hat{y}^{**} = 0.39447 \cdot 10 + 1.1260 = 5.0707$$

Surprisingly to me, the grades predicted by both models appear to be the same, approximately 5 out of 7. Of course, I would see minimal deviations between the two grades if I used more decimal places in the parameters but this is redundant considering that my aim was only to approximate the optimal parameters m and c. In any case, the aim of this exploration has been successfully achieved as I have optimized the parameters for the linear regression line for my data using the gradient descent algorithm.

2.3 Limitations of Gradient Descent

As I mentioned earlier, gradient descent is a slow algorithm and it requires a lot of iterations to converge. Each iteration requires computation and time, so the more iterations are needed, the slower and more computationally expensive the algorithm becomes. In fact, the algorithm converges to the optimal parameters $(0.39447, 1.1260)$ correct to five significant figures after 2017 iterations [Appendix B] which in more sophisticated problems is a lot of computing power. The speed of convergence is proportional to the learning rate $\alpha \in [0, 1]$ as it scales the step size yet it does not allow great flexibility.

The gradient descent algorithm converges to points where the slope is zero as then the new position does not change:

$$\begin{pmatrix} m_{i+1} \\ c_{i+1} \end{pmatrix} = \begin{pmatrix} m_i \\ c_i \end{pmatrix} - \alpha \cdot 0 = \begin{pmatrix} m_i \\ c_i \end{pmatrix}$$

I realized that this is a limitation since the points with zero slope are not only minimum points but also maximum points and stationary inflexion points (or saddle points in three dimensions). If the learning rate is chosen correctly, gradient descent is guaranteed to minimize the function (within some tolerance), so getting stuck at maximum points is not a problem. However, if the initial position is set above a stationary inflexion point of a 2D function, gradient descent might get stuck there as the slope will be very close to zero stopping any further progress. Also, if a function has several minima one of which is global, gradient descent is not guaranteed to find the coordinates of the global minimum. This is shown in the figure below.

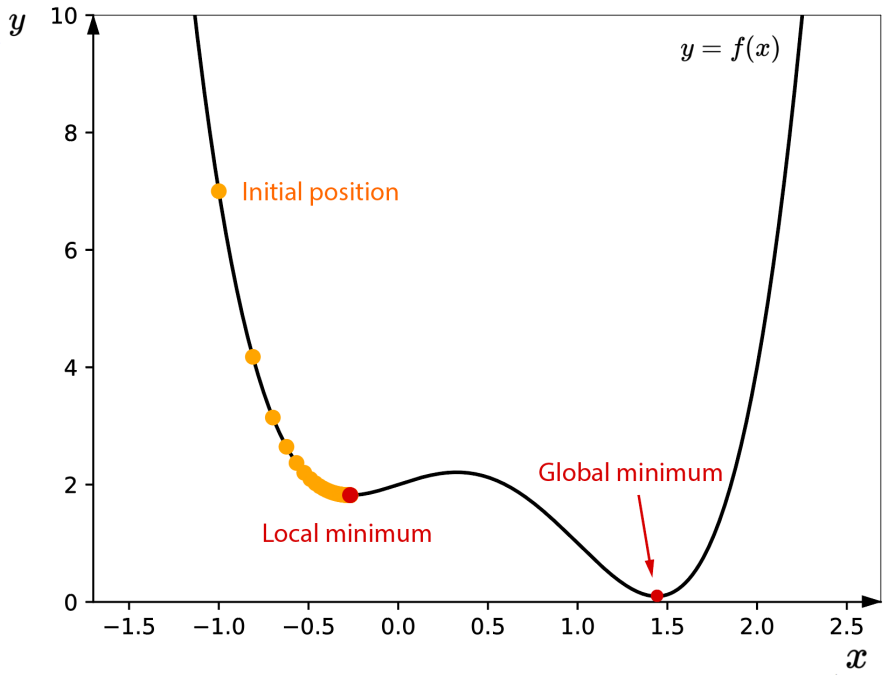

Figure 7: Gradient descent run on an arbitrary function with a local minimum and a global minimum

Imagine the following continuous differentiable function that has a local minimum and a global minimum. If the initial point is set to $(-1, 7)$, the algorithm will converge to the local minimum and will get stuck there. The algorithm will never reach the global minimum and it is a problem since the parameters will not be optimized as much as they can be. Of course, I could try different starting points like where $x = 2$ but this would assume that I know what the function looks like. In the cases where there are more than two parameters for a machine learning model, the graphs would be in more than three dimensions which is very hard to visualize. A real-world case where the impact of these limitations can be seen is with neural networks. Below is an attempt by computer scientists to visualize the so-called loss landscape, or a plot of the loss function (like my $J(m, c)$), of a neural network called ResNet-110 [12].

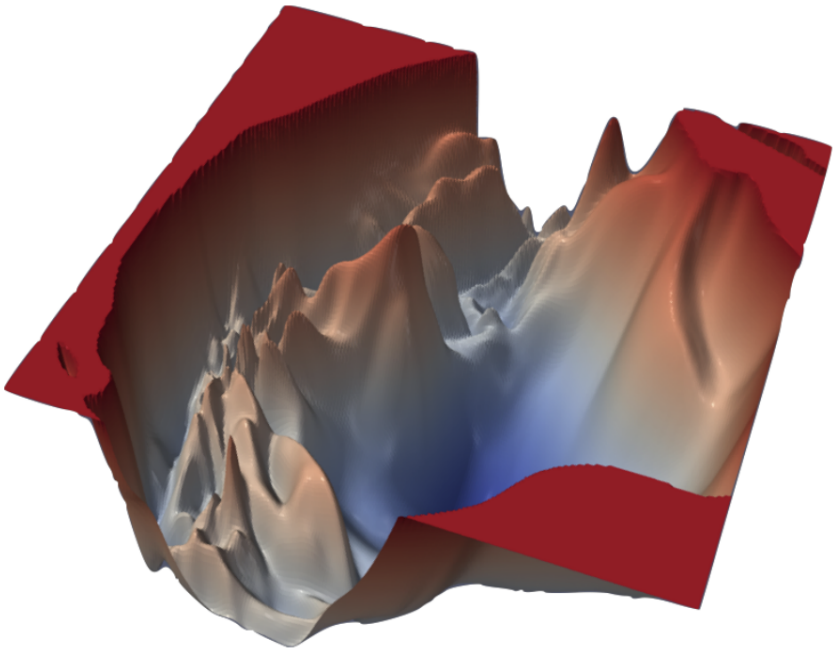

Figure 8: Loss landscape of a neural network [12]

This loss landscape has a myriad of local minima and saddle points and gradient descent is very unlikely to get to the global minimum on the first try. Scientists can try different starting points and learning rates but this, of course, is not efficient. Luckily, gradient descent can be optimised using special optimisers like AdaGrad [13], Adam [14] and Momentum Acceleration [15] which are a topic for a separate exploration.

3 Conclusion

The aim of my exploration was to show how gradient descent can be used in a linear regression problem and in the general case of approximating the minimum points of multivariable functions. I have successfully achieved this aim. During the process of my investigation, I stumbled upon some peculiar features of gradient descent that limit its usability in real-world problems and I discussed them at the end of my exploration.

One limitation of my exploration is that due to page limit, I did not touch upon convex and non-convex functions that ultimately determine whether regular gradient descent will be effective or not. Also, using a larger dataset for a more important cause than my hypothetical exam grade would add to the real-world significance of my exploration.

However, this was not my main aim. Some other topics that I found exciting to explore are the ways to optimize gradient descent using optimisers, as well as its other types such as stochastic gradient descent and mini-batch gradient descent.

Appendices

```python
def loss_function(m, c):
    return ((1-(m+c))**2 + (2-(3*m+c))**2 + (3-(4*m+c))**2 + (4-(8*m+c))**2 + (5-(14*m+c))**2+(6-(9*m+c))**2 + (7-(12*m+c))**2)/7

def djdm(m, c): # m, θ
    return (-2/7)*((1-(m+c))*(1) + (2-(3*m+c))*(3) + (3-(4*m+c))*(4) + (4-(8*m+c))*(8) + (5-(14*m+c))*(14) + (6-(9*m+c))*(9) + (7-(12*m+c))*(12))

def djdc(m, c): # θ, c
    return (-2/7)*((1-(m+c)) + (2-(3*m+c)) + (3-(4*m+c)) + (4-(8*m+c)) + (5-(14*m+c)) + (6-(9*m+c)) + (7-(12*m+c)))

r_min, r_max = -5, 1
m = np.arange(-3, 3, 0.5)
c = np.arange(-5, 5, 0.5)
M, C = np.meshgrid(m, c)
J = loss_function(M, C)


ax = plt.subplot(projection='3d', computed_zorder=False)

# initial position and learning rate
start_m = 1
start_c = -3
current_pos = [start_m, start_c, loss_function(start_m, start_c)]
learning_rate = 0.01
MAX_STEPS = 4000

solutions_m = [start_m]
solutions_c = [start_c]
scores = [loss_function(start_m, start_c)]

# initial conditions (position and loss)
print(f'θ | m = {current_pos[0]} | c = {current_pos[1]} | MSE = {loss_function(current_pos[0], current_pos[1])}')

# setting up the plots
fig2, ax_contour = plt.subplots(1,1)
norm = PowerNorm(vmin=J.min(), vmax=J.max(), gamma=0.5)
cp = ax_contour.contourf(M, C, J, 60, cmap='YlGnBu_r', norm=norm)
fig2.colorbar(cp, label='loss J') # Add a colorbar to a plot

# titles and lables of the plots
ax_contour.set_title('Contour plot of the loss function')
ax_contour.set_xlabel('slope m')
ax_contour.set_ylabel('intercept c')

# plotting starting points
ax.scatter(current_pos[0], current_pos[1], current_pos[2], color='orange', zorder=3)
ax_contour.scatter(current_pos[0], current_pos[1], color='orange', zorder=3)
```

Appendix A.1: The first part of my code of the gradient descent algorithm in Python programming language

```python
for i in range(MAX_STEPS):
    # plot the points on 3d plot
    ax.scatter(current_pos[0], current_pos[1], current_pos[2], color='white', zorder=2)
    # plot the points on contour plot
    ax_contour.scatter(current_pos[0], current_pos[1], current_pos[2], color='white', zorder=2)

    # update rule: calculate derivatives and new positions
    d_dm, d_dc = djdm(current_pos[0], current_pos[1]), djdc(current_pos[0], current_pos[1])
    new_m, new_c = current_pos[0] - learning_rate * d_dm, current_pos[1] - learning_rate * d_dc
    new_j = loss_function(new_m, new_c)

    # append to lists in order to plot the steps on 3d plot
    solutions_m.append(new_m)
    solutions_c.append(new_c)
    scores.append(loss_function(new_m, new_c))
    current_pos = (new_m, new_c, new_j)

    ax.plot(solutions_m, solutions_c, scores, '-', linewidth=0.1, color='lightgrey', zorder=1)


    # print the parameters m and c after a number of iterations
    if (i+1) in [1, 2, 100, 500, 1000]:
        print(f'{i+1} | m = {new_m} | c = {new_c} | MSE = {"{:.5f}".format(loss_function(new_m, new_c))}')+

    # plot the 3D function and the contour plot
    if i == (MAX_STEPS - 1):
        ax.plot_surface(M, C, J, cmap='YlGnBu_r', norm=norm, zorder=0)
        ax.scatter(current_pos[0], current_pos[1], current_pos[2], color='aquamarine', zorder=3)
        ax.scatter(slope_ideal, intercept_ideal, loss_function(slope_ideal, intercept_ideal), color='red', zorder=4)
        ax.set_xlabel('slope m')
        ax.set_ylabel('intercept c')
        ax.set_zlabel('loss J')

        ax_contour.scatter(current_pos[0], current_pos[1], current_pos[2], color='aquamarine', zorder=2)
        ax_contour.scatter(slope_ideal, intercept_ideal, loss_function(slope_ideal, intercept_ideal), color='red', zorder=4)
        ax_contour.plot(solutions_m, solutions_c, '-', linewidth=0.7, color='lightgrey', zorder=1)

        print(f'slope: {round(new_m, 5)} | intercept: {round(new_c,5)}')


plt.savefig('3d_GD_contour_plot.svg')
plt.show()
```

Appendix A.2: The second part of my code of the gradient descent algorithm in Python programming language

```
2014 | m = 0.3944747916212868 | c = 1.1259489375105214 | MSE = 0.90061
2015 | m = 0.39447475067410354 | c = 1.1259493462669234 | MSE = 0.90061
2016 | m = 0.3944747099481607 | c = 1.125949752814787 | MSE = 0.90061
2017 | m = 0.39447 | c = 1.1260 | MSE = 0.90061
```

Appendix B: Screenshot of the algorithm output for iterations 2014-2017

References

[1] IBM, "What is machine learning?," 2023.

https://www.ibm.com/topics/machine-learning.

[2] N. Donges, "Gradient descent in machine learning: A basic introduction," *Built In*.

https://builtin.com/data-science/gradient-descent.

[3] Python *Python.org*. https://www.python.org/.

[4] scikit learn, "scikit-learn: machine learning in python," *Scikit-learn.org*.

https://scikit-learn.org/stable/.

[5] V. Kanade, "What is linear regression? types, equation, examples, and best practices for 2022," *Spiceworks*, April 2023. https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-linear-regression/.

[6] I. Sanober, "Loss functions in deep learning," *InsideAIML*. https://insideaiml.com/blog/LossFunctions-in-Deep-Learning-1025.

[7] A. Gupta, "Mean squared error: Overview, examples, concepts and more," *Simplilearn.com*, February 2023. https://www.simplilearn.com/tutorials/statistics-tutorial/mean-squared-error.

[8] J. Brownlee, "Analytical vs numerical solutions in machine learning," *Machine Learning Mastery*, March 2018. https://machinelearningmastery.com/analytical-vs-numerical-solutions-in-machine-learning/.

[9] "Vector calculus: Understanding the gradient – betterexplained." https://betterexplained.com/articles/vector-calculus-understanding-the-gradient/.

[10] J. Brownlee, "Understand the impact of learning rate on neural network performance," *Machine Learning Mastery*, Sepember 2020. https://machinelearningmastery.com/understand-the-dynamics-of-learning-rate-on-deep-learning-neural-networks/: :text=Learning

[11] "Contour plots: Definition, examples," *Statistics How to*. https://www.statisticshowto.com/contour-plots/.

[12] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, "Visualizing the loss landscape of neural nets," November 2018. https://arxiv.org/abs/1712.09913.

[13] J. Duchi, E. Hazan, and Y. Singer, *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*, vol. 12. 2011. https://www.jmlr.org/papers/volume12/duchi11a/duchi11a.pdf.

[14] D. Kingma and J. Lei Ba, *Adam: a Method for Stochastic Optimization*. December 2014. https://arxiv.org/pdf/1412.6980.pdf.

[15] B. Polyak, *Some methods of speeding up the convergence of iteration methods*. September 1964. https://www.mathnet.ru/links/714458d1b66bf61d0c9ecc7fa05f98b4/zvmmf7713.pdf.